



ITI - Apache Web Server

Content



1

Web Server
Fundamentals

2

Apache
Web Server

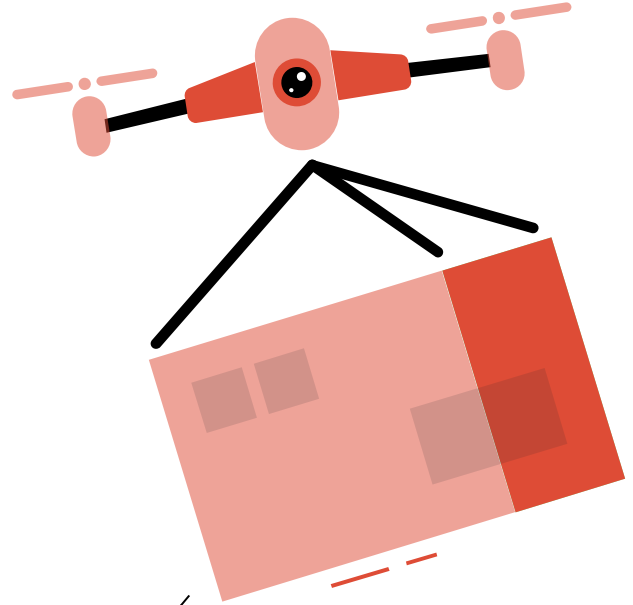
3

Apache Components

4

Apache
Demo

Web Server Fundamentals



Web Server Fundamentals

1

How The Website Works?

2

What is DNS?

3

HTTP vs HTTPS

4

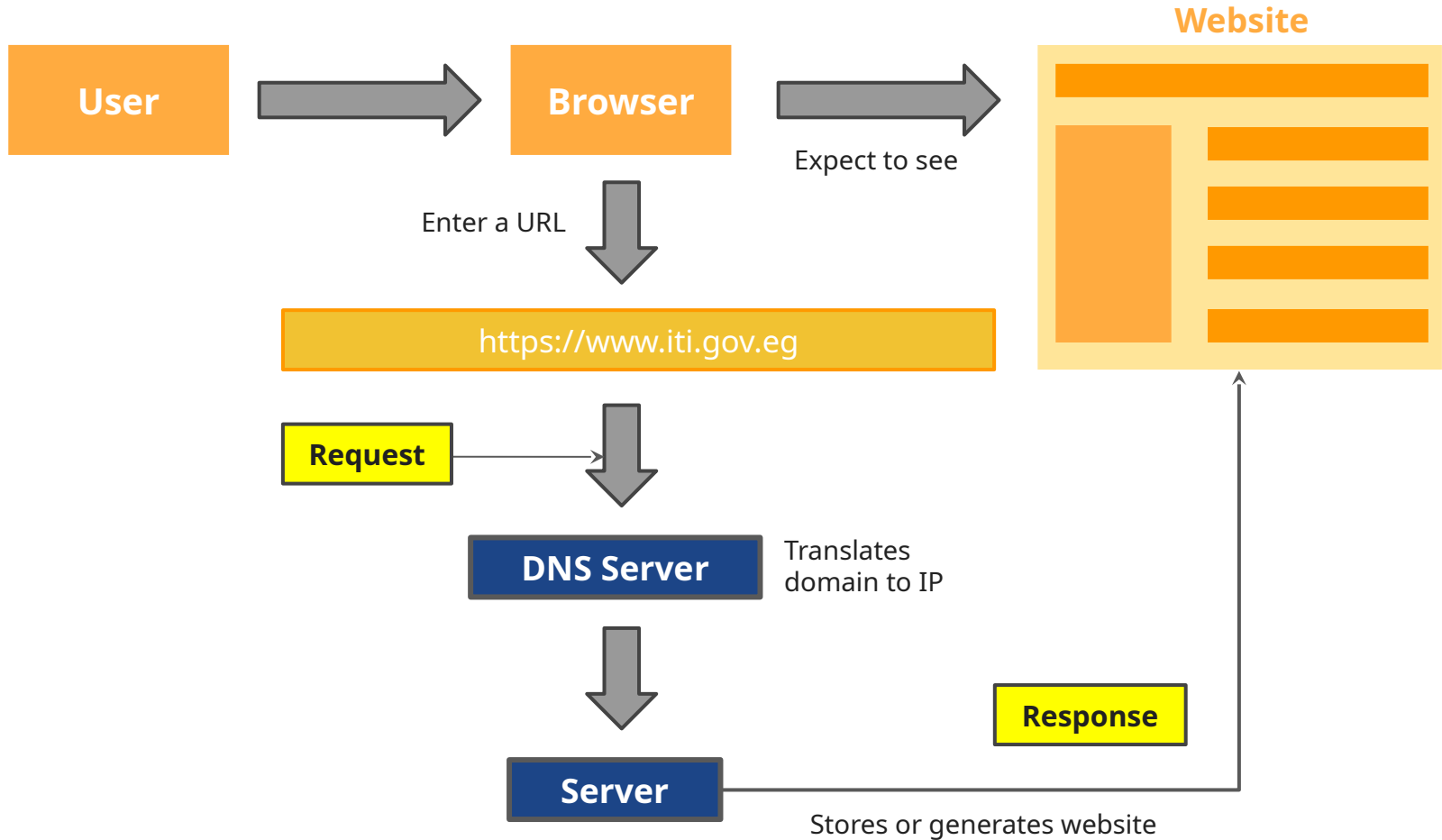
HTTP Status Codes

5

Anatomy of Get & Post Requests

6

What's Web Servers?





What's the DNS?

- The Domain Name System (DNS) is the **phonebook** of the Internet. Humans access information online through domain names, like **google.com**. Web browsers interact through Internet Protocol (IP) addresses. DNS **translates** domain names to IP addresses so browsers can load Internet resources.
- Each device connected to the Internet has a **unique IP address** which other machines use to find the device.

HTTP Definition



Hypertext Transfer Protocol (HTTP) is an application-layer protocol for transmitting hypermedia documents, such as HTML. It was designed for communication between **web browsers** and **web servers**.

HTTP	HTTPS
URL begins with http://	URL begins with https://
The request is processed through port 80	The request is processed through port 443
Fast transfer of unencrypted data over a simpler protocol	Longer data transfer, as there are additional steps to provide encryption
Main purpose - data exchange on the internet	Main purpose - confidential data exchange
Does not improve search ranking	Improves search ranking

HTTP Methods

- **GET**

Retrieves data from the server

- **POST**

Submit data to the server

- **PUT**

Update data already on the server

- **Delete**

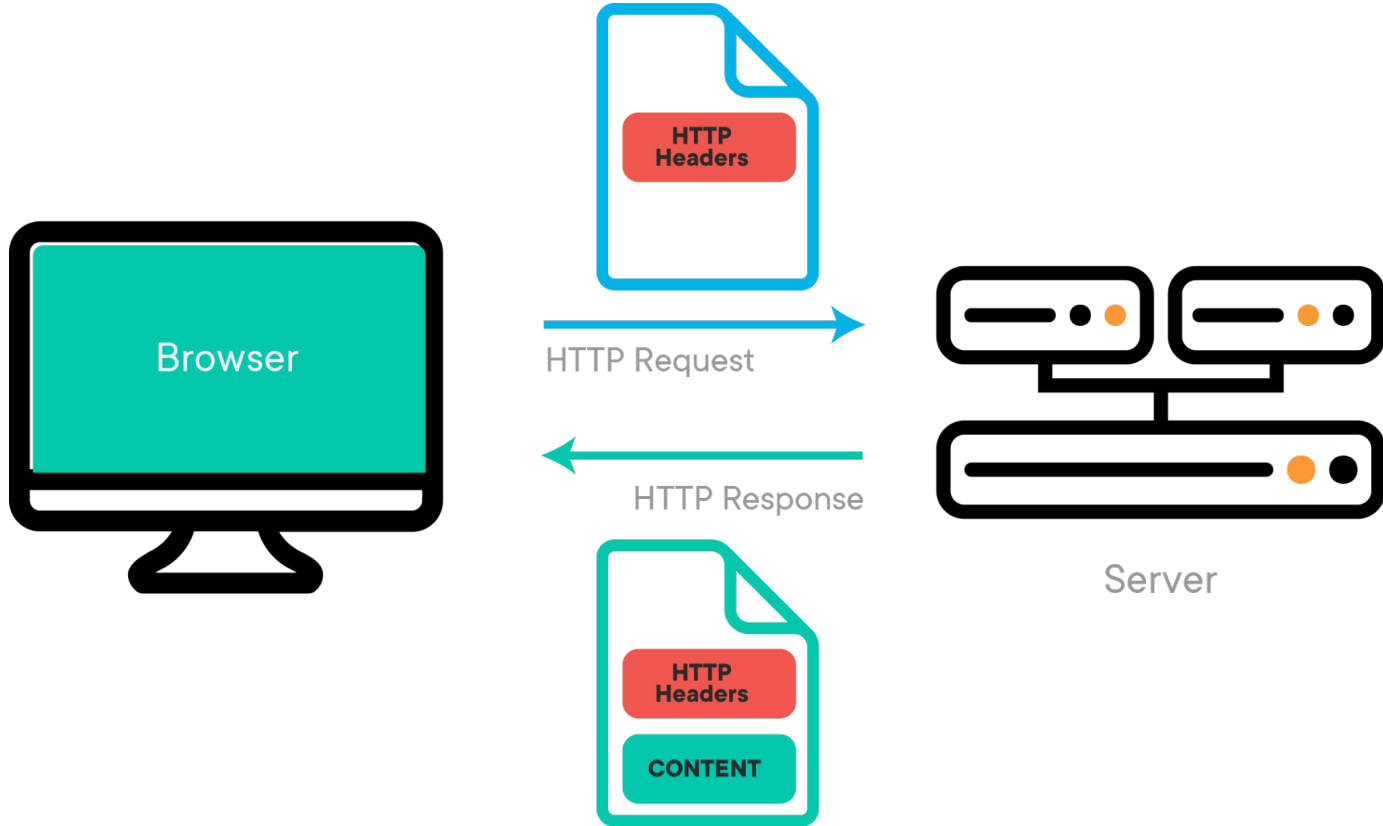
Deletes data from the server

HTTP Status Codes

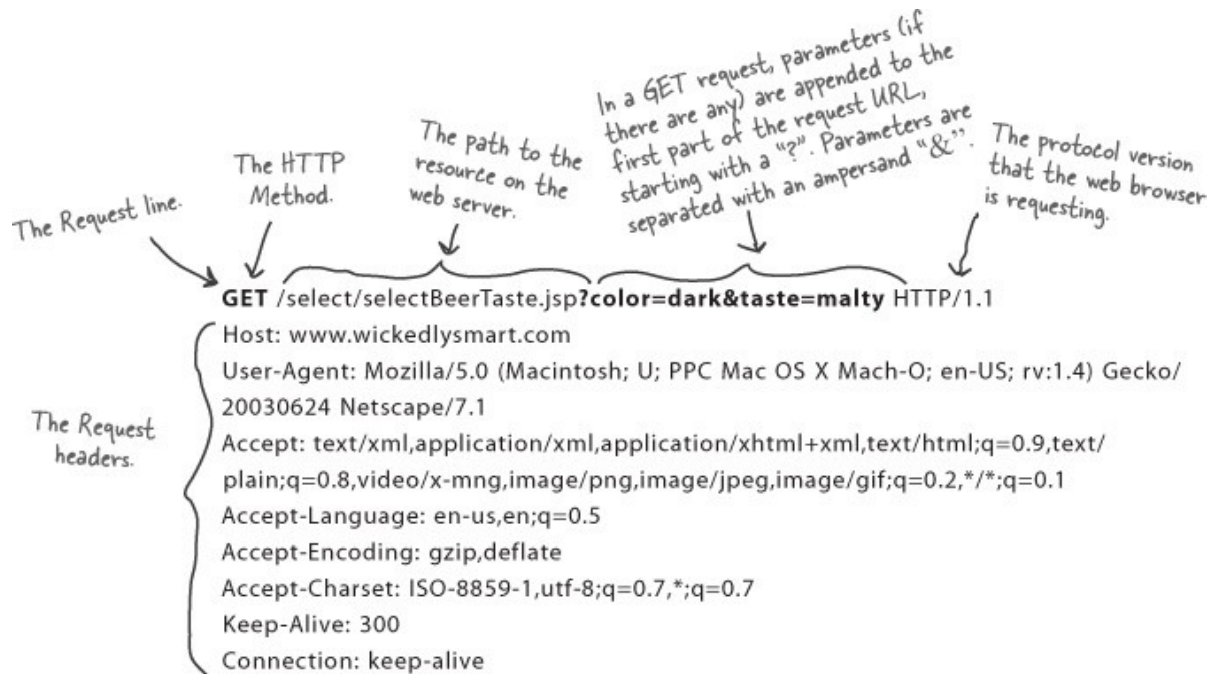
- **1xx : Informational**
Request received / Processing
- **2xx : Success**
Successfully received
- **3xx : Redirect**
Further action must be taken
- **4xx : Client Error**
Request doesn't have what it needs
- **5xx : Server Error**
Server fulfil the request

HTTP Status Codes

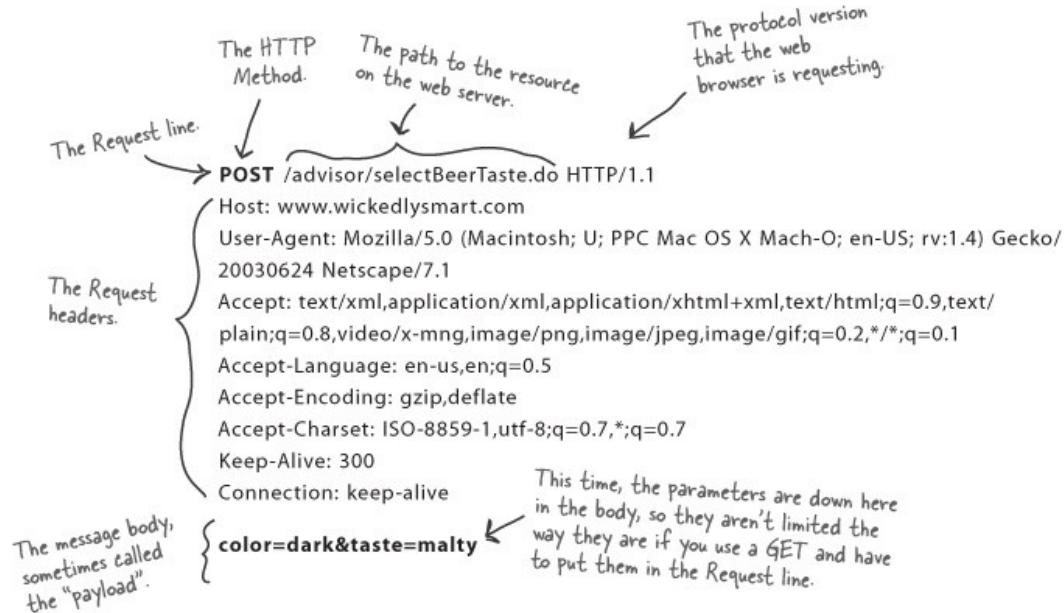
- **200** - Ok
- **201** - Ok created
- **301** - Moved to new URL
- **304** - Not modified (Cached)
- **400** - Bad request
- **401** - Unauthorized
- **404** - Not found
- **500** - Internal server error



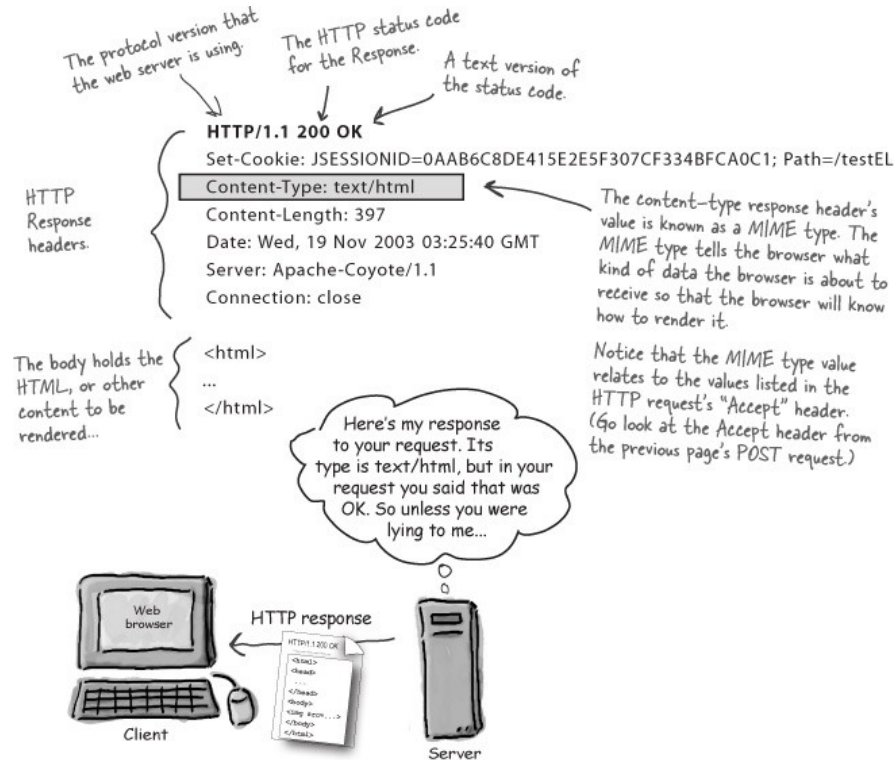
GET Request Anatomy



POST Request Anatomy



Response Anatomy

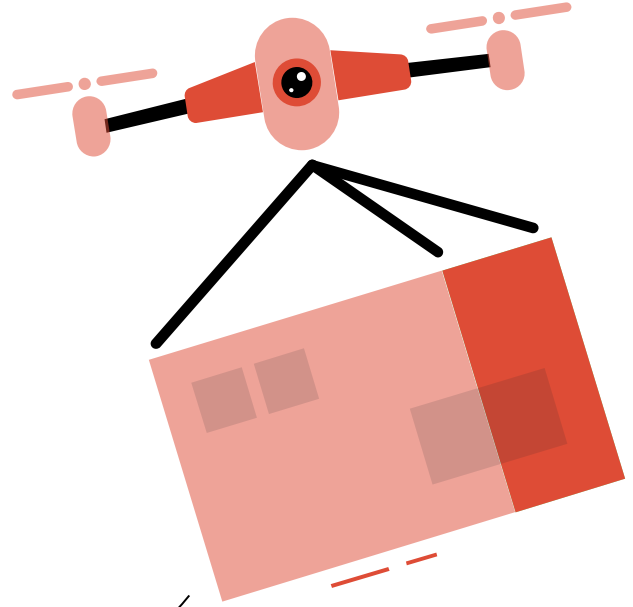


What's Web Server ?



- This is a **computer** that's sole purpose is to **distribute** information that is hosted within its hard drives.
- Depending on the information, it is accessed and distributed **differently**.
- A detailed example would be standard Web Pages that are accessed via the Internet protocol **HTTP on port 80** and distributed back in the same fashion which are stored on a web server.

Apache Web Server



Apache History



1994

Developed by Rob McCool at the national center for supercomputing applications



1995

Brian Behlendorf and Cliff Skolnick with others continued the development



1999

Apache Software Foundation formed to provide organizational, legal and financial support for the Apache HTTP server

Apache Web Server



Apache HTTP Server is a **free and open-source** web server that delivers web content through the internet.

It quickly became the most popular HTTP client on the web. It was revealed that the name originated from the respect of the **Native American tribe** for its resiliency and durability.

Apache Pros



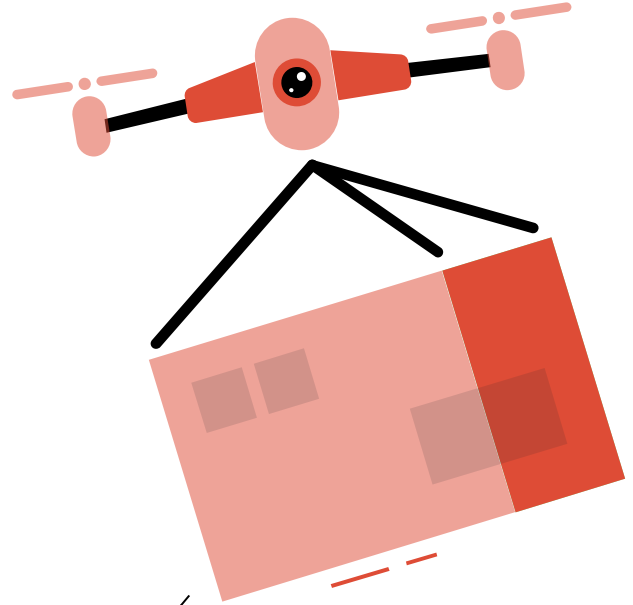
- Open-source and **free**, even for commercial use.
- Reliable, **stable** software.
- Frequently updated **security** patches.
- **Flexible** due to its module-based structure.
- Easy to configure, **beginner-friendly**.
- **Cross-platform** (works on both Unix and Windows servers).
- Optimal deliverability for **static files** and compatibility with any programming language (PHP, Python.. etc)
- Huge community and easily available **support** in case of any problem.

Apache Cons



- Performance problems on extremely traffic-heavy websites.
- Too many configuration options can lead to security vulnerabilities.

Apache Components



Configuration Files



- The Apache HTTP Server is configured via **simple text files**.
- The default **configuration** file is usually called **httpd.conf**.
- The configuration is frequently broken into **multiple smaller files**, for ease of management. These files are loaded via the Include directive.

Main Configuration File



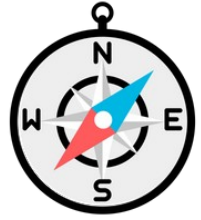
- Ubuntu:

`/etc/apache2/apache2.conf`

- RedHat or CentOS:

`/etc/httpd/conf/httpd.conf`

Directives



- The server is configured by placing **configuration directives** in configuration files.
- A directive is a **keyword** followed by one or more arguments that set its value.
- In addition to the main configuration files, certain directives may go in **.htaccess** files located in the **content directories**.
.htaccess files are primarily for people who do not have access to the main server configuration file(s)

Public Web Files

- Ubuntu:

`/var/www/html`

- RedHat or CentOS:

`/var/www/html`

Directives Examples

- **ServerRoot** directive

The path to the server's configuration.

- **PidFile** directive

The process identification number for the httpd registered at starting the server.

- **ServerName** directive

This is where you declare the name of your website.

Directives Examples - Cont.

- **DocumentRoot** directive

This is where your web documents (html files, images etc) should be located.

- **Listen** directive

The Listen directive instructs Apache httpd to listen to only specific IP addresses or ports

Directives Contexts

- **Server config:** This means that the directive may be used in the server configuration files (e.g., httpd.conf), but not within any `<VirtualHost>` or `<Directory>` containers. It is not allowed in `.htaccess` files at all.
- **Virtual host :** This context means that the directive may appear inside `<VirtualHost>` containers in the server configuration files. **Directory :** A directive marked as being valid in this context may be used inside `<Directory>`, `<Location>`, `<Files>`, and `<Proxy>` containers in the server configuration files, subject to the restrictions outlined in Configuration Sections.
- **.htaccess:** If a directive is valid in this context, it means that it can appear inside per-directory `.htaccess` files. It may not be processed, though depending upon the overrides currently active.

AllowOverride Directive

Description:	Types of directives that are allowed in .htaccess files
Syntax:	<code>AllowOverride All None directive-type [directive-type] ...</code>
Default:	<code>AllowOverride None</code> (2.3.9 and later), <code>AllowOverride All</code> (2.3.8 and earlier)
Context:	directory
Status:	Core
Module:	core

When the server finds an .htaccess file (as specified by `AccessFileName`) it needs to know which directives declared in that file can override earlier configuration directives.

KeepAlive Directive

Description:	Enables HTTP persistent connections
Syntax:	KeepAlive On Off
Default:	KeepAlive On
Context:	server config, virtual host
Status:	Core
Module:	core

The Keep-Alive extension to HTTP/1.0 and the persistent connection feature of HTTP/1.1 provide long-lived HTTP sessions which allow multiple requests to be sent over the same TCP connection. In some cases this has been shown to result in an almost 50% speedup in latency times for HTML documents with many images. To enable Keep-Alive connections, set KeepAlive On

MaxKeepAliveRequests Directive

Description: Number of requests allowed on a persistent connection

Syntax: MaxKeepAliveRequests *number*

Default: MaxKeepAliveRequests 100

Context: server config, virtual host

Status: Core

Module: core

The MaxKeepAliveRequests directive limits the number of requests allowed per connection when KeepAlive is on. If it is set to 0, unlimited requests will be allowed. We recommend that this setting be kept to a high value for maximum server performance

ErrorLog Directive

<u>Description:</u>	Format specification for error log entries
<u>Syntax:</u>	ErrorLogFormat [connection request] <i>format</i>
<u>Context:</u>	server config, virtual host
<u>Status:</u>	Core
<u>Module:</u>	core

The ErrorLog directive sets the name of the file to which the server will log any errors it encounters. If the file-path is not absolute then it is assumed to be relative to the ServerRoot

Directory Directive

Description: Enclose a group of directives that apply only to the named file-system directory, sub-directories, and their contents.

Syntax: `<Directory directory-path> ... </Directory>`

Context: server config, virtual host

Status: Core

Module: core

`<Directory>` and `</Directory>` are used to enclose a group of directives that will apply only to the named directory, sub-directories of that directory, and the files within the respective directories. Any directive that is allowed in a directory context may be used.

DirectoryMatch Directive

Description:	Enclose directives that apply to the contents of file-system directories matching a regular expression.
Syntax:	<code><DirectoryMatch regex> ... </DirectoryMatch></code>
Context:	server config, virtual host
Status:	Core
Module:	core

`<DirectoryMatch>` and `</DirectoryMatch>` are used to enclose a group of directives which will apply only to the named directory (and the files within), the same as `<Directory>`. However, it takes as an argument a regular .expression.

Files Directive

Description:	Contains directives that apply to matched filenames
Syntax:	<code><Files filename> ... </Files></code>
Context:	server config, virtual host, directory, .htaccess
Override:	All
Status:	Core
Module:	core

The `<Files>` directive limits the scope of the enclosed directives by filename. It is comparable to the `<Directory>` and `<Location>` directives. It should be matched with a `</Files>` directive. The directives given within this section will be applied to any object with a basename (last component of filename) matching the specified filename

FilesMatch Directive

Description: Contains directives that apply to regular-expression matched filenames

Syntax: `<FilesMatch regex> ... </FilesMatch>`

Context: server config, virtual host, directory, .htaccess

Override: All

Status: Core

Module: core

The `<FilesMatch>` directive limits the scope of the enclosed directives by filename, just as the `<Files>` directive does. However, it accepts a regular expression.

Allow Directive

Description: Controls which hosts can access an area of the server

Syntax: Allow from all|host|env=[!]env-variable [host|env=[!]env-variable] ...

Context: directory, .htaccess

Override: Limit

Status: Extension

Module: mod_access_compat

The Allow directive affects which hosts can access an area of the server. Access can be controlled by hostname, IP address, IP address range, or by other characteristics of the client request captured in environment variables

Deny Directive

Description:	Controls which hosts are denied access to the server
Syntax:	Deny from all <i>host</i> env=[!] <i>env-variable</i> [<i>host</i> env=[!] <i>env-variable</i>] ...
Context:	directory, .htaccess
Override:	Limit
Status:	Extension
Module:	mod_access_compat

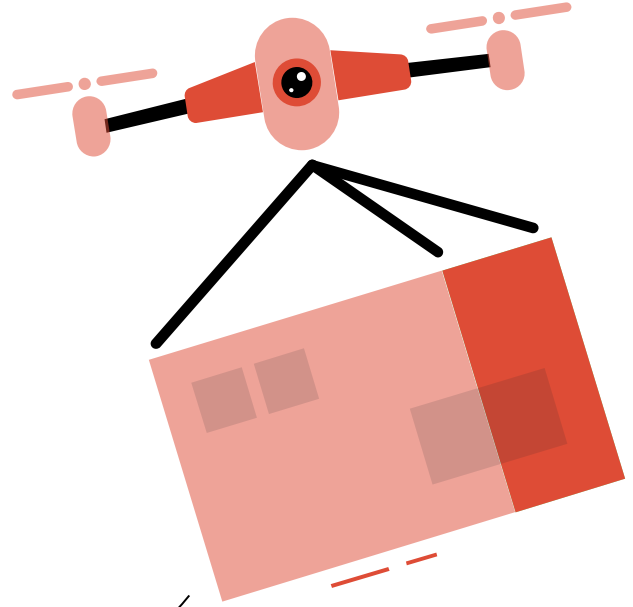
This directive allows access to the server to be restricted based on hostname, IP address, or environment variables. The arguments for the Deny directive are identical to the arguments for the Allow directive

CacheEnable Directive

Description:	Enable caching of specified URLs using a specified storage manager
Syntax:	CacheEnable <i>cache_type</i> [<i>url-string</i>]
Context:	server config, virtual host, directory
Status:	Extension
Module:	mod_cache
Compatibility:	A url-string of '/' applied to forward proxy content in 2.2 and earlier.

The CacheEnable directive instructs mod_cache to cache urls at or below url-string. The cache storage manager is specified with the cache_type argument. The CacheEnable directive can alternatively be placed inside either <Location> or <LocationMatch> sections to indicate the content is cacheable

Apache Demo



.htaccess for AUTHENTICATION

- **.htaccess content**

```
AuthType Basic
```

```
AuthName "Restricted web page"
```

```
AuthUserFile "/var/www/html/.htpasswd"
```

```
require valid-user
```

- **To create .htpasswd use this command**

```
htpasswd -c .htpasswd username
```

.htaccess for AUTHENTICATION

- **AuthType Basic** defines the type of authentication.
- **Basic** means there is no encryption and the password hash is sent as clear text. This is one of the major reasons why .htaccess cannot be considered for protection of confidential user data.
- **"Restricted web page"** is a window title string. When someone tries to access an .htaccess-protected page, a username & password window will pop in the web browser. This window will bear a title - this is the AuthName. It can be anything you like.
- **AuthUserFile** /var/www/.htpasswd defines the path to a file where user credentials are stored. This file does not exist, but we will create it soon.
- **require valid-user** indicates only successful authentication attempts will result in the loading of the page.

Apache Lab



- Install Apache HTTP server
- Create two simple html pages named “page1.html, page2.html”
- then use the `suitable` directive to automatically redirect from `localhost/page1.html` to `localhost/page2.html`.
- Ask for username and password when accessing a directory.
- Create a directory then allow access to one of your classmates only.
- Change the default index page to be `default.html` instead of `index.html`

Thanks!

Do you have any questions?